There's almost always a way to isolate critical code into an idempotent routine, and usually the two techniques we just discussed will let you do it. If there isn't, and retrying the job is unsafe, you'll need to prevent the job from retrying. You can do this on a per-job basis by calling `sidekiq_options` in the job, like so:

```
class JobThatCannotBeIdempotent
  include Sidekiq::Job
  sidekiq_options retry: 0

  def peform(...)
    # ...
  end
end
```

The first time that job fails, it'll go to the dead set. Your configured death handler will alert you, and you can take whatever manual action you need to decide what to do.

We'll talk more about thinking through idempotency in a bit, but before we do that, we should make sure we understand how to, or if we even can, test all this.